SahiReport - Security Audit & Remediation

Technical Analysis, Step-by-Step Fixes & Hardening Guide

Date: November 2026 | Scope: Client Application, Supabase Backend & Platform Manifests

1. Executive Summary

SahiReport underwent a comprehensive security review covering mobile client logic, backend database access controls (Row-Level Security), cryptographic payment verification, GPS tamper protection, and app store compliance standards. All identified vulnerabilities have been remediated across the codebase.

This document provides technical documentation of each flaw, the attack vector mitigated, the exact source files modified, and verification procedures.

2. Vulnerability & Remediation Matrix

ID	Severity	Category	Vulnerability Description	Status
1.1	CRITICAL	Payment Security	Client-side plan upgrade / Payment bypass	RESOLVED
1.2	CRITICAL	API Authorization	Unauthenticated FCM push notification Edge Function	RESOLVED
2.1	HIGH	Data Integrity	GPS-verified spoofing via Mock Location apps	RESOLVED
2.2	HIGH	Access Control	Co-Owner authentication & RLS session mismatch	RESOLVED
2.3	HIGH	Authentication	Low entropy on admin access codes (brute-force)	RESOLVED
3.1	MEDIUM	Storage Security	Anonymous storage uploads without MIME validation	RESOLVED
3.2	MEDIUM	Compliance	Incomplete account deletion (App Store Guideline 5.1.1)	RESOLVED
3.3	MEDIUM	Store Policy	Unused ACCESS_BACKGROUND_LOCATION in Android	RESOLVED
3.4	MEDIUM	Compliance	Placeholder legal & privacy policy URLs	RESOLVED
4.1	LOW	User Experience	Registration redirect mismatch (Email link vs SMS)	RESOLVED
4.2	LOW	Storage Leak	Orphaned storage image files on project deletion	RESOLVED

3. Critical Security Fixes

3.1 Server-Side Payment Signature Verification (Payment Bypass)

CRITICAL VULNERABILITY MITIGATED

- Problem: When Razorpay completed on the client, billing_screen.dart directly executed an UPDATE query on the owners table to grant pro_project_slots without any server-side cryptographic proof.
- Risk: Any user with the Supabase anon key could craft a direct database request to grant themselves unlimited annual Pro project slots without ever paying.
- Remediation: Created verify-payment Supabase Edge Function that computes HMAC-SHA256 signature using RAZORPAY_KEY_SECRET. If the signature matches razorpay_signature, the function updates the database using the service_role key.
- Database Trigger: Added PostgreSQL trigger trg_protect_owner_subscription_fields blocking any direct client updates to plan_type, pro_project_slots, or pro_expires_at.

```
// verify-payment/index.ts HMAC-SHA256 validation
const signaturePayload = `${razorpay_order_id}|${razorpay_payment_id}`;
const cryptoKey = await crypto.subtle.importKey('raw', encoder.encode(secret), ...);
const signatureBuffer = await crypto.subtle.sign('HMAC', cryptoKey, payload);
if (!timingSafeEqual(expectedSignature, razorpay_signature)) {
  return json({ error: 'Invalid payment signature. Possible fraud attempt.' }, 400);
}
```

3.2 Securing send-notification Edge Function

CRITICAL VULNERABILITY MITIGATED

- Problem: The send-notification Edge Function used the service-role key but did not check incoming Authorization headers, allowing public invocation from any HTTP client.
- Remediation: Added strict Supabase Bearer JWT verification, validating token structure and signature before accepting submission_id or triggering Google Firebase FCM messages.

4. High-Severity Security Fixes

4.1 GPS Tamper Protection & Mock Location Detection

HIGH SEVERITY MITIGATED

- Problem: The app claimed 'GPS-Verified' daily progress reports, but never inspected pos.isMocked from geolocator. Users could use Fake GPS apps to submit false reports from home.
- Remediation: Added mandatory pos.isMocked validation in field_home_screen.dart. If mock location is active, submission is blocked with an alert.

```
final pos = await Geolocator.getCurrentPosition(desiredAccuracy: LocationAccuracy.high);
if (pos.isMocked) {
  setState(() { _gpsError = 'Mock location detected. Please disable Fake GPS.'; });
  return;
}
```

4.2 Access Code Entropy Upgrade (Brute-Force Prevention)

HIGH SEVERITY MITIGATED

- Problem: Co-Owner administrative access codes previously used 4 random characters (ADMN-XXXX), yielding under 925,000 combinations. An attacker could enumerate codes in hours.
- Remediation: Expanded suffix to 8 random alphanumeric characters (ADMN-XXXXXXXX), yielding over 850 billion combinations. Added rate limiting in auth_rate_limits table.

5. Regulatory Compliance & Platform Hardening

5.1 Storage Bucket MIME and File Size Constraints

- Hardening: Configured storage.buckets policy for submission-images enforcing a 5 MB maximum size limit and restricting MIME types to image/jpeg, image/png, and image/webp.

5.2 App Store Account Deletion Compliance (Apple Guideline 5.1.1)

- Requirement: Apple requires in-app account deletion to purge all user data, not just set a soft-delete flag.
- Remediation: Wired softDeleteAccount() in auth_provider.dart to invoke delete-account Edge Function. Added daily pg_cron job purge_soft_deleted_accounts() to automatically wipe data older than 30 days.

5.3 Native Platform Configuration Updates (Android & iOS)

- Android Manifest: Removed unused ACCESS_BACKGROUND_LOCATION permission. Added FCM default notification channel (sahi_report_submissions). Added <queries> for url_launcher.
- Android ProGuard: Added proguard-rules.pro keeping Razorpay JavaScript interfaces (@JavascriptInterface), AdMob, and Firebase reflection classes.
- iOS Info.plist: Removed NSLocationAlwaysAndWhenInUseUsageDescription to prevent App Store rejection. Added CFBundleURLTypes for fieldtrack:// deep linking. Expanded SKAdNetworkItems.
- iOS Podfile: Fixed uppercase GCC_PREPROCESSOR_DEFINITIONS for permission_handler.

6. Workflow & Storage Optimization

- Registration OTP Flow: Fixed owner_register_screen.dart to redirect users to ConfirmEmailScreen rather than expecting an SMS OTP that was never dispatched.
- Storage Cleanup: Updated deleteProject() in ProjectRepository to collect all image paths from submission_images and purge them from Supabase Storage before dropping the project row.

7. Deployment & Verification Checklist

- Step 1 (Database): Open Supabase Dashboard -> SQL Editor and execute supabase/migrations/20261102_security_hardening.sql.
- Step 2 (Edge Functions): Run: supabase secrets set RAZORPAY_KEY_SECRET='your_secret' && supabase functions deploy verify-payment && supabase functions deploy send-notification.
- Step 3 (Android Build): Execute: flutter build appbundle --release.
- Step 4 (iOS Build): Execute: cd ios && pod install && cd .. && flutter build ipa --release.